

Inżynieria danych

Wykład 6: organizacja plików II

Katarzyna Grygiel

Semestr letni 2025/2026

Organizacja krotek w pliku

Plik stanowi kolekcję stron, z których każda zawiera pewną liczbę rekordów. Na pliku wykonywane są następujące operacje:

- wstawianie/usuwanie/modyfikowanie rekordu,
- odczytywanie konkretnego rekordu,
- wyszukiwanie wszystkich rekordów spełniających zadane warunki.

Organizacja krotek w pliku

Plik stanowi kolekcję stron, z których każda zawiera pewną liczbę rekordów. Na pliku wykonywane są następujące operacje:

- wstawianie/usuwanie/modyfikowanie rekordu,
- odczytywanie konkretnego rekordu,
- wyszukiwanie wszystkich rekordów spełniających zadane warunki.

Na podstawie organizacji rekordów możemy wyróżnić (między innymi) następujące rodzaje plików:

- plik nieuporządkowany,
- plik uporządkowany według klucza,
- klasteryzacja wielotabelowa,
- B+-drzewa,
- plik haszowy.

Pliki nieuporządkowane

W przypadku plików nieuporządkowanych rekordy mogą być przechowywane w dowolnym miejscu. Po wstawieniu rekordu najczęściej nie zmienia on już swojego położenia.

Rekord może być wstawiany zawsze na końcu pliku. Ponieważ jednak usunięcie rekordu zwalnia miejsce, dobrym rozwiązaniem jest jego późniejsze wykorzystanie przy wstawianiu nowego rekordu.

Pliki nieuporządkowane

W przypadku plików nieuporządkowanych rekordy mogą być przechowywane w dowolnym miejscu. Po wstawieniu rekordu najczęściej nie zmienia on już swojego położenia.

Rekord może być wstawiany zawsze na końcu pliku. Ponieważ jednak usunięcie rekordu zwalnia miejsce, dobrym rozwiązaniem jest jego późniejsze wykorzystanie przy wstawianiu nowego rekordu.

W takiej sytuacji ważne jest, aby SZBD potrafił efektywnie znaleźć wolne miejsce, bez konieczności sekwencyjnego przeszukiwania wszystkich stron w pliku.

Pliki nieuporządkowane

W przypadku plików nieuporządkowanych rekordy mogą być przechowywane w dowolnym miejscu. Po wstawieniu rekordu najczęściej nie zmienia on już swojego położenia.

Rekord może być wstawiany zawsze na końcu pliku. Ponieważ jednak usunięcie rekordu zwalnia miejsce, dobrym rozwiązaniem jest jego późniejsze wykorzystanie przy wstawianiu nowego rekordu.

W takiej sytuacji ważne jest, aby SZBD potrafił efektywnie znaleźć wolne miejsce, bez konieczności sekwencyjnego przeszukiwania wszystkich stron w pliku.

Większość systemów korzysta ze struktury danych zwanej **mapą wolnego miejsca** (*free-space map*), w której zapisane są informacje o dostępności miejsca na danej stronie.

Mapa wolnego miejsca

Mapa wolnego miejsca jest najczęściej reprezentowana za pomocą tablicy, której każdy element odpowiada jednej stronie i określa, jaka część strony jest wolna.

W PostgreSQL możemy odczytać rozmiar dostępnej części dla każdej ze strony z dokładnością do 32 bajtów.

Mapa wolnego miejsca

Mapa wolnego miejsca jest najczęściej reprezentowana za pomocą tablicy, której każdy element odpowiada jednej stronie i określa, jaka część strony jest wolna.

W PostgreSQL możemy odczytać rozmiar dostępnej części dla każdej ze strony z dokładnością do 32 bajtów.

demo

```
CREATE TABLE tab AS
  SELECT * FROM generate_series(1,1000) AS n;
DELETE FROM tab WHERE mod(n, 2) = 0;
SELECT ctid, * FROM tab;
SELECT * FROM pg_freespace('tab');
VACUUM FULL tab;
SELECT * FROM pg_freespace('tab');
```


Mapa wolnego miejsca

Aby znaleźć stronę dla nowego rekordu o zadanym rozmiarze, można przeszukać mapę wolnego miejsca, poszukując odpowiednio dużej strony. Gdy takiej strony nie ma, dla relacji przydzielana jest nowa.

Mapa wolnego miejsca

Aby znaleźć stronę dla nowego rekordu o zadanym rozmiarze, można przeszukać mapę wolnego miejsca, poszukując odpowiednio dużej strony. Gdy takiej strony nie ma, dla relacji przydzielana jest nowa.

Ponieważ dla dużych plików taka operacja może być wolna, mapę można rozszerzyć do drzewa binarnego, w którym w liściach przechowywane są tablice z informacjami o dostępności dla ustalonej liczby stron, natomiast w węzłach wewnętrznych znajduje się maksymalna wartość przechowywana w niższych węzłach.

Mapa wolnego miejsca

Aby znaleźć stronę dla nowego rekordu o zadany rozmiarze, można przeszukać mapę wolnego miejsca, poszukując odpowiednio dużej strony. Gdy takiej strony nie ma, dla relacji przydzielana jest nowa.

Ponieważ dla dużych plików taka operacja może być wolna, mapę można rozszerzyć do drzewa binarnego, w którym w liściach przechowywane są tablice z informacjami o dostępności dla ustalonej liczby stron, natomiast w węzłach wewnętrznych znajduje się maksymalna wartość przechowywana w niższych węzłach.

Aktualizacja mapy po każdej modyfikacji zawartości relacji byłaby kosztowna, dlatego wykonuje się ją co jakiś czas. Dlatego informacje zawarte w mapie mogą być w danej chwili nieaktualne.

Wady i zalety plików nieuporządkowanych

Zalety:

- szybkie wstawianie krotek,
- efektywne przy odczycie wszystkich krotek,
- sprawdza się przy stosunkowo małych plikach.

Wady:

- przy wyszukiwaniu krotki może być konieczne przeszukanie całego pliku.

Pliki uporządkowane sekwencyjnie

W plikach uporządkowanych sekwencyjnie rekordy są ustawione według pewnego **klucza wyszukiwania** (*search key*). Pozwala to na ich szybsze przetwarzanie.

Kluczem wyszukiwania może być atrybut lub zbiór atrybutów, nie musi to być żaden z (super)kluczy danej relacji. Aby odczytywać rekordy w zadanym porządku, dodajemy do każdego rekordu wskaźnik wskazujący na następny rekord.

Pliki uporządkowane sekwencyjnie

W plikach uporządkowanych sekwencyjnie rekordy są ustawione według pewnego **klucza wyszukiwania** (*search key*). Pozwala to na ich szybsze przetwarzanie.

Kluczem wyszukiwania może być atrybut lub zbiór atrybutów, nie musi to być żaden z (super)kluczy danej relacji. Aby odczytywać rekordy w zadanym porządku, dodajemy do każdego rekordu wskaźnik wskazujący na następny rekord.

Aby zminimalizować liczbę operacji dostępu do strony, rekordy przechowywane są fizycznie w miarę możliwości w kolejności odpowiadającej żadanemu porządkowi.

Pliki uporządkowane sekwencyjnie

W plikach uporządkowanych sekwencyjnie rekordy są ustawione według pewnego **klucza wyszukiwania** (*search key*). Pozwala to na ich szybsze przetwarzanie.

Kluczem wyszukiwania może być atrybut lub zbiór atrybutów, nie musi to być żaden z (super)kluczy danej relacji. Aby odczytywać rekordy w zadanym porządku, dodajemy do każdego rekordu wskaźnik wskazujący na następny rekord.

Aby zminimalizować liczbę operacji dostępu do strony, rekordy przechowywane są fizycznie w miarę możliwości w kolejności odpowiadającej żądanemu porządkowi.

Taka organizacja rekordów umożliwia uporządkowany odczyt, co jest przydatne przy wyświetlaniu wyniku oraz przy przetwarzaniu zapytań. Trudniejszymi operacjami są natomiast operacje wstawiania i usuwania rekordów.

Pliki uporządkowane sekwencyjnie: usuwanie rekordu

<i>studenci</i>			
id	imię	kierunek	
103	Anna	informatyka	↓
124	Ewa	biologia	↓
254	Iwo	psychologia	↓
267	Jan	matematyka	↓
305	Lena	matematyka	↓
372	Leon	filozofia	↓
405	Maja	filozofia	↓

Pliki uporządkowane sekwencyjnie: usuwanie rekordu

studenci

id	imię	kierunek	
103	Anna	informatyka	↓
124	Ewa	biologia	
254	Łukasz	psychologia	↓
267	Jan	matematyka	↓
305	Lena	matematyka	↓
372	Leon	filozofia	↓
405	Maja	filozofia	↓

Pliki uporządkowane sekwencyjnie: wstawianie rekordu

<i>studenci</i>		
id	imię	kierunek
103	Anna	informatyka
124	Ewa	biologia
267	Jan	matematyka
305	Lena	matematyka
372	Leon	filozofia
405	Maja	filozofia
254	Iwo	psychologia

Pliki uporządkowane sekwencyjnie: wstawianie rekordu

studenci

id	imię	kierunek
103	Anna	informatyka
124	Ewa	biologia
267	Jan	matematyka
305	Lena	matematyka
372	Leon	filozofia
405	Maja	filozofia
254	Iwo	psychologia



Wady i zalety plików uporządkowanych

Zalety:

- efektywny odczyt krotek według atrybutów porządkujących,
- łatwe znalezienie następnej krotki według atrybutów porządkujących,
- szybkie wyszukiwanie według atrybutów porządkujących (metodą połowienia binarnego).

Wady:

- nieefektywny odczyt krotek według atrybutów nieporządkujących,
- kosztowne wstawianie i usuwanie krotek, spowodowane koniecznością zachowania uporządkowania.

Klasteryzacja wielotabelowa

Czasami wygodnie jest przechowywać rekordy z kilku relacji w jednym bloku. Jest to przydatne w sytuacji, gdy zapytania dotyczą w przeważającej mierze operacji na złączeniach tabel.

Klasteryzacja wielotabelowa

Czasami wygodnie jest przechowywać rekordy z kilku relacji w jednym bloku. Jest to przydatne w sytuacji, gdy zapytania dotyczą w przeważającej mierze operacji na złączeniach tabel.

Rozważmy następujące zapytanie oparte na tabelach *pracownicy*(id, nazwisko, id_w) oraz *wydziały*(id, nazwa, adres):

```
SELECT id, nazwisko, w.id, nazwa, adres  
FROM pracownicy JOIN wydziały w ON (id_w = w.id);
```

Dla każdej krotki z tabeli *pracownicy* system musi znaleźć krotkę o odpowiedniej wartości **id** w tabeli *wydziały*.

Klasteryzacja wielotabelowa

pracownicy

id	nazwisko	id_w
10	Einstein	1
12	Strzelecki	3
26	Platon	2
30	Humboldt	3
37	Kelvin	1
40	Newton	1

wydziały

id	nazwa	adres
1	Fizyki	Piękna 7
2	Filozofii	Ładna 8
3	Nauk o Ziemi	Wesoła 17

Klasteryzacja wielotabelowa

pracownicy

id	nazwisko	id_w
10	Einstein	1
12	Strzelecki	3
26	Platon	2
30	Humboldt	3
37	Kelvin	1
40	Newton	1

wydziały

id	nazwa	adres
1	Fizyki	Piękna 7
2	Filozofii	Ładna 8
3	Nauk o Ziemi	Wesoła 17

1	Fizyki	Piękna 7	10	Einstein	1
37	Kelvin	1	40	Newton	1
2	Filozofii	Ładna 8	26	Platon	2
3	Nauk o Ziemi	Wesoła 17	12	Strzelecki	3
30	Humboldt	3			

Klasteryzacja wielotabelowa

To, czy korzystać z klasteryzacji wielotabelowej, zależy od typu zapytań, które są najczęściej wykonywane. Zastosowanie takiej organizacji rekordów może znacząco przyspieszyć przetwarzanie zapytań.

Klasteryzacja wielotabelowa

To, czy korzystać z klasteryzacji wielotabelowej, zależy od typu zapytań, które są najczęściej wykonywane. Zastosowanie takiej organizacji rekordów może znacząco przyspieszyć przetwarzanie zapytań.

Łatwo wskazać przykład zapytania, w przypadku którego taka organizacja znacznie spowolni jego wykonanie:

```
SELECT * FROM wydziały;
```

W takiej sytuacji należy przejrzeć istotnie więcej bloków, aby znaleźć rekordy zawierające informacje o wydziałach. Można co prawda przechowywać dodatkowo wskaźniki, za pomocą których łatwiejsze stanie się znalezienie rekordów dla odpowiedniej relacji, nie zmniejszy to jednak w znaczącym stopniu liczby przeglądanych stron.

Pliki haszowe

Funkcja haszująca (*hash function*) oblicza wartość na argumencie nazywanym **kluczem haszowanym** (*hash key*), którym może być dowolny ustalony atrybut lub ciąg atrybutów danej relacji (najlepiej jeden z kluczy).

Pliki haszowe

Funkcja haszująca (*hash function*) oblicza wartość na argumencie nazywanym **kluczem haszowanym** (*hash key*), którym może być dowolny ustalony atrybut lub ciąg atrybutów danej relacji (najlepiej jeden z kluczy).

Wartościami funkcji haszującej są liczby całkowite z przedziału $[0, B - 1]$, gdzie B jest liczbą **kubeków** (*bucket*).

Przez kubelek rozumiemy zbiór rekordów, dla których funkcja haszująca przyjmuje tę samą wartość. Funkcja haszująca dla różnych rekordów może zwracać tę samą wartość – dochodzi wtedy do **kolizji** (*collision*).

Pliki haszowe

Funkcja haszująca (*hash function*) oblicza wartość na argumencie nazywanym **kluczem haszowanym** (*hash key*), którym może być dowolny ustalony atrybut lub ciąg atrybutów danej relacji (najlepiej jeden z kluczy).

Wartościami funkcji haszującej są liczby całkowite z przedziału $[0, B - 1]$, gdzie B jest liczbą **kubelków** (*bucket*).

Przez kubełek rozumiemy zbiór rekordów, dla których funkcja haszująca przyjmuje tę samą wartość. Funkcja haszująca dla różnych rekordów może zwracać tę samą wartość – dochodzi wtedy do **kolizji** (*collision*).

W **pliku haszowym** (*hash file*) porządek rekordów jest wyznaczony przez funkcję haszującą, której wartości odpowiadają adresom stron, na których powinien znajdować się dany rekord.

Funkcje haszujące: szybkość i kolizje

Dobra funkcja haszująca powinna dla zbioru losowo wybranych danych wejściowych zapełnić kubeczki w możliwie równomierny sposób.

Funkcje haszujące: szybkość i kolizje

Dobra funkcja haszująca powinna dla zbioru losowo wybranych danych wejściowych zapełnić kubeczki w możliwie równomierny sposób.

Najszybciej działającą funkcją haszującą jest funkcja stała: wszystkie argumenty znajdują się w jednym kubeczku.

Na drugim biegunie znajduje się doskonała funkcja haszująca, która dla każdego klucza zwróci inną wartość.

Funkcje haszujące: szybkość i kolizje

Dobra funkcja haszująca powinna dla zbioru losowo wybranych danych wejściowych zapełnić kubeczki w możliwie równomierny sposób.

Najszybciej działającą funkcją haszującą jest funkcja stała: wszystkie argumenty znajdują się w jednym kubeczku.

Na drugim biegunie znajduje się doskonała funkcja haszująca, która dla każdego klucza zwróci inną wartość.

Szukamy kompromisu pomiędzy szybkością haszowania, a współczynnikiem kolizji. Nie zależy nam na tym, aby funkcja haszująca była dobra z kryptograficznego punktu widzenia (np. funkcje MD5 i SHA są za wolne).

Przykładowe funkcje haszujące

Często dobrym wyborem jest funkcja

$$h(x) = x \bmod p,$$

gdzie p jest liczbą pierwszą (i odpowiada liczbie kubeków).

Przykładowe funkcje haszujące

Często dobrym wyborem jest funkcja

$$h(x) = x \bmod p,$$

gdzie p jest liczbą pierwszą (i odpowiada liczbie kubelków).

Inne stosowane funkcje haszujące:

- CRC (używana do wykrywania błędów przy transferze danych),
- MurmurHash (szybka funkcja ogólnego przeznaczenia),
- Google CityHash (szybka dla krótkich kluczy, do 64 bajtów),
- Google FarmHash (nowsza wersja CityHash, mniej kolizji),
- Facebook xxHash (daje obecnie najlepsze wyniki).

Benchmark

Metoda adresowania otwartego (próbkiowanie liniowe)

Dla n kluczy tworzymy tablicę z haszowaniem o dużym rozmiarze, np. $2n$.

Problem kolizji rozwiązujemy przydzielając nowemu rekordowi pierwszy wolny kubełek.

Metoda adresowania otwartego (próbkiwanie liniowe)

Dla n kluczy tworzymy tablicę z haszowaniem o dużym rozmiarze, np. $2n$.

Problem kolizji rozwiązujemy przydzielając nowemu rekordowi pierwszy wolny kubełek.

Podczas szukania, obliczamy wartość funkcji haszującej na odpowiednim kluczu i sprawdzamy, czy wskazany kubełek zawiera poszukiwany rekord. Jeśli nie, to zaglądamy do następnego kubeczka itd. Szukanie kończymy po natrafieniu na pusty kubełek.

Metoda adresowania otwartego (próbkiowanie liniowe)

Dla n kluczy tworzymy tablicę z haszowaniem o dużym rozmiarze, np. $2n$.

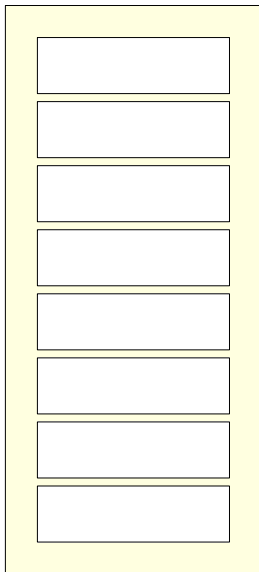
Problem kolizji rozwiązujemy przydzielając nowemu rekordowi pierwszy wolny kubełek.

Podczas szukania, obliczamy wartość funkcji haszującej na odpowiednim kluczu i sprawdzamy, czy wskazany kubełek zawiera poszukiwany rekord. Jeśli nie, to zaglądamy do następnego kubeczka itd. Szukanie kończymy po natrafieniu na pusty kubełek.

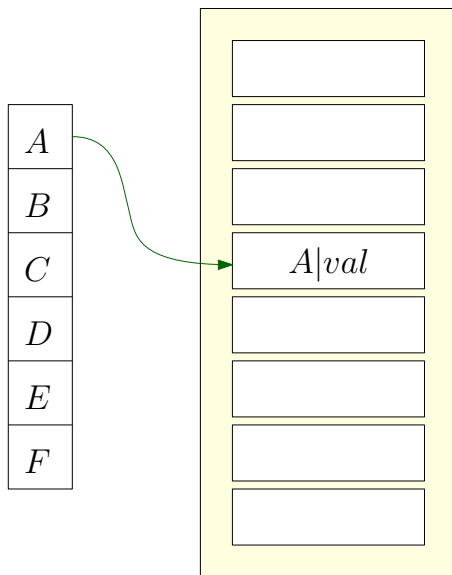
W związku z tym należy sprytnie podejść do operacji usuwania, nie możemy bowiem zostawić pustego kubeczka. Mamy dwie możliwości: albo przenosimy część rekordów do kubeczków „wyżej”, albo zostawiamy odpowiedni znacznik („tu byłem”).

Próbkowanie liniowe: wstawianie

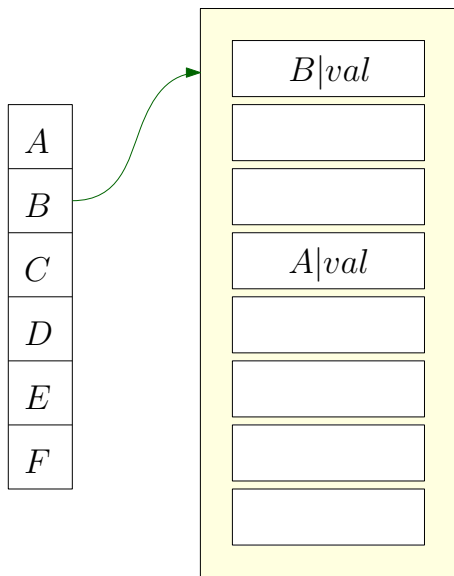
<i>A</i>
<i>B</i>
<i>C</i>
<i>D</i>
<i>E</i>
<i>F</i>



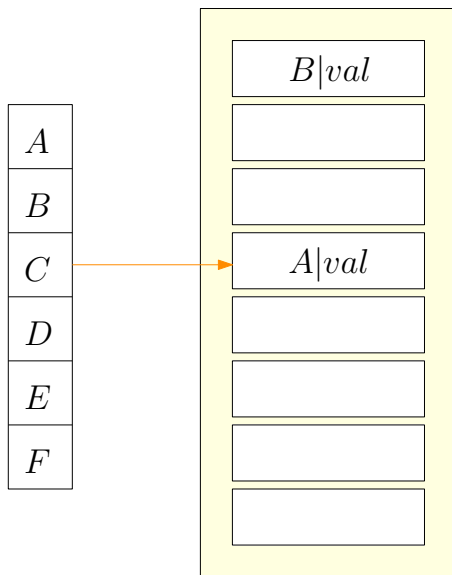
Próbkowanie liniowe: wstawianie



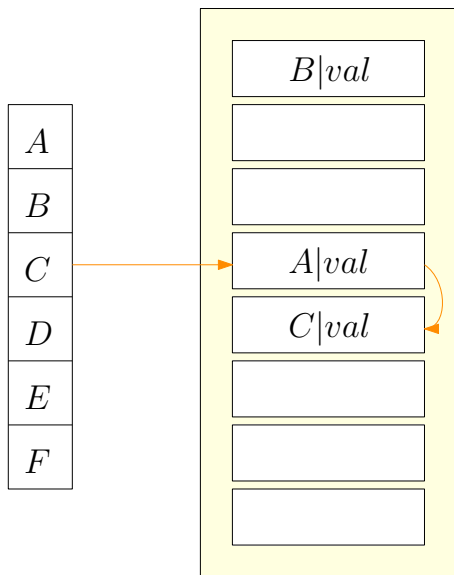
Próbkowanie liniowe: wstawianie



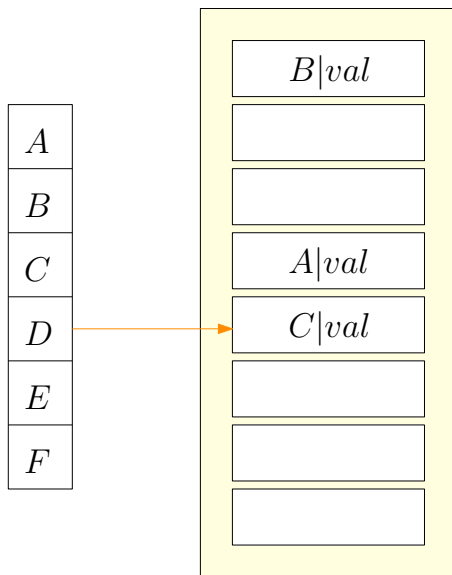
Próbkowanie liniowe: wstawianie



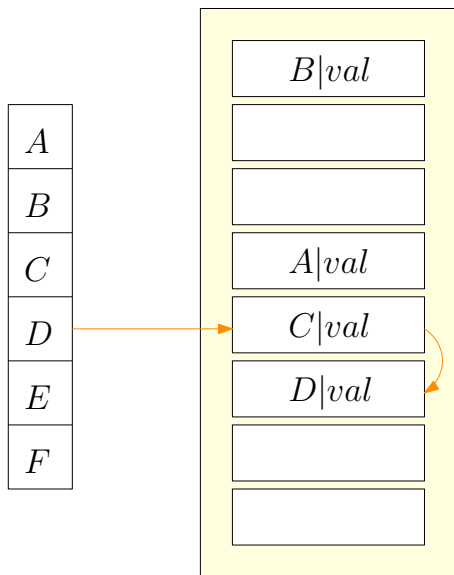
Próbkowanie liniowe: wstawianie



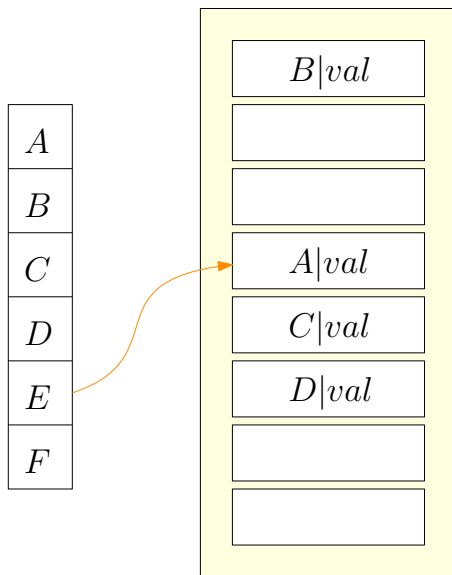
Próbkowanie liniowe: wstawianie



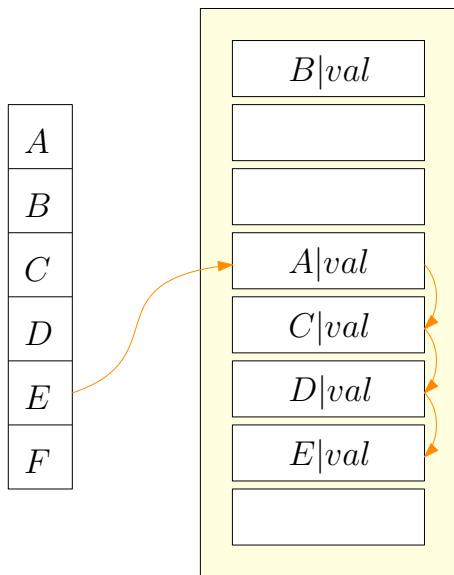
Próbkowanie liniowe: wstawianie



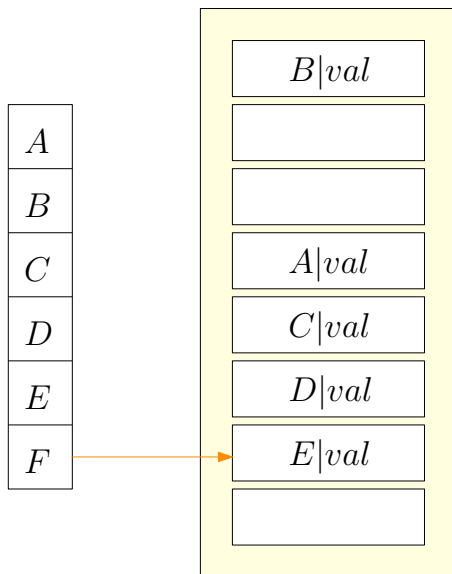
Próbkowanie liniowe: wstawianie



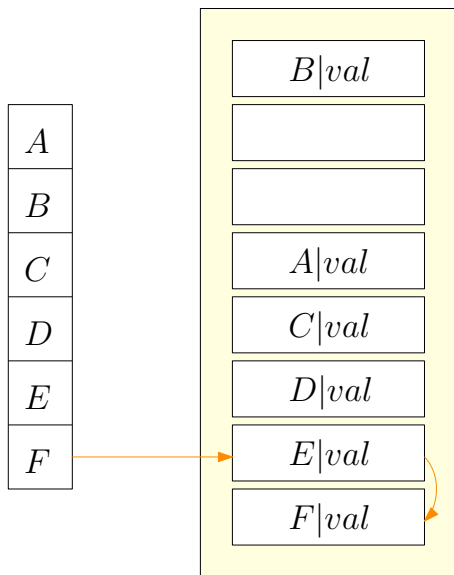
Próbkowanie liniowe: wstawianie



Próbkowanie liniowe: wstawianie



Próbkowanie liniowe: wstawianie

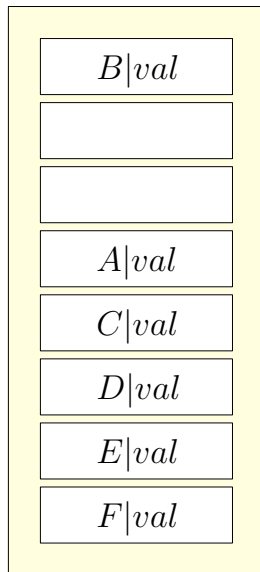
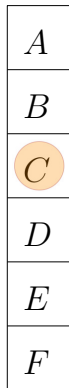


Próbkowanie liniowe: wstawianie

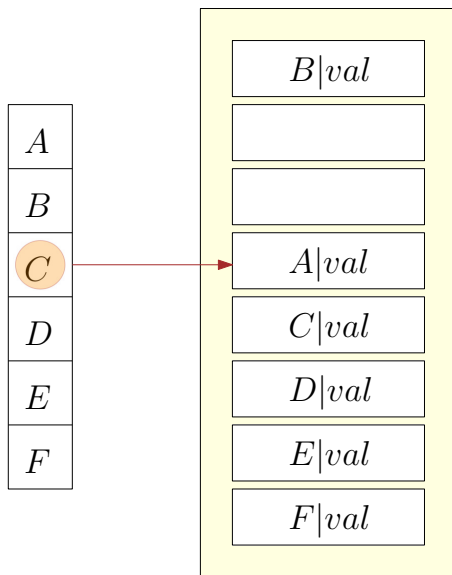
A
B
C
D
E
F

$B val$
$A val$
$C val$
$D val$
$E val$
$F val$

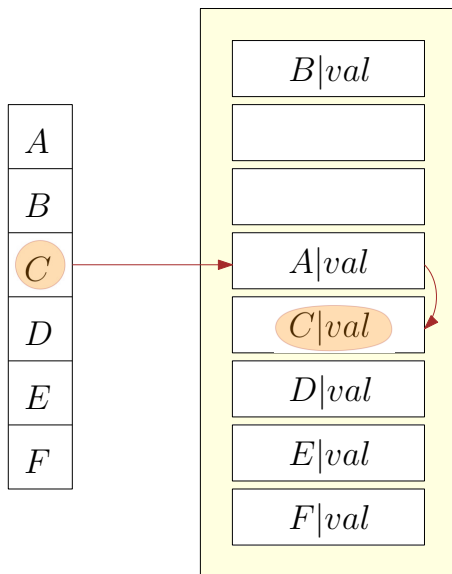
Próbkowanie liniowe: usuwanie



Próbkowanie liniowe: usuwanie



Próbkowanie liniowe: usuwanie



Próbkowanie liniowe: usuwanie

A
B
C
D
E
F

$B val$
$A val$
$D val$
$E val$
$F val$

Próbkowanie liniowe: usuwanie

A
B
C
D
E
F

$B val$
$A val$
?
$D val$
$E val$
$F val$

Próbkowanie liniowe: usuwanie

Wersja 1

A
B
C
D
E
F



Próbkowanie liniowe: usuwanie

Wersja 2

A
B
C
D
E
F

$B val$
$A val$
$D val$
$E val$
$F val$

Próbkowanie liniowe Robin Hooda

Próbkowanie liniowe możemy ulepszyć stosując **haszowanie Robin Hooda** (*Robin Hood hashing*).

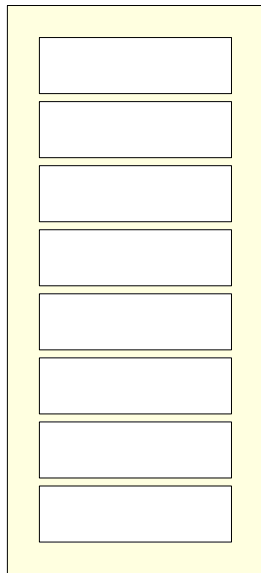
Wraz z kluczem K zapisujemy liczbę pozycji $r(K)$, która nas dzieli od optymalnego miejsca w tablicy.

Jeżeli podczas operacji wstawiania dochodzi do kolizji, to klucz K zajmuje miejsce innego klucza K' wtedy i tylko wtedy, gdy $r(K) > r(K')$, czyli gdy znajduje się dalej od swojego optymalnego miejsca niż klucz już wstawiony.

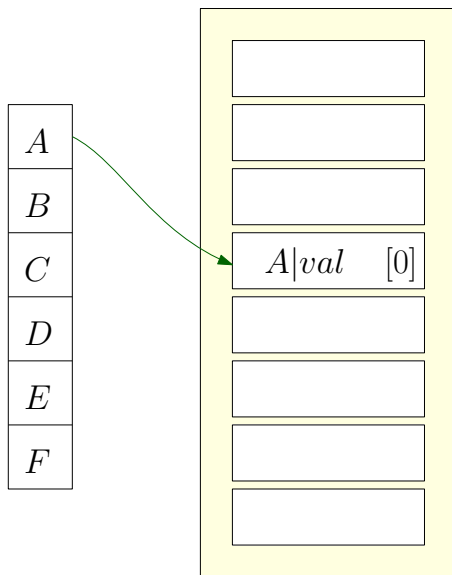
W wersji Robin Hooda haszowanie zabiera miejsca „bogatym” kluczom (czyli tym, które są blisko właściwego miejsca) i przydziela je „biednym” kluczom (które umieszczone są daleko od swoich optymalnych pozycji).

Próbkowanie liniowe Robin Hooda: wstawianie

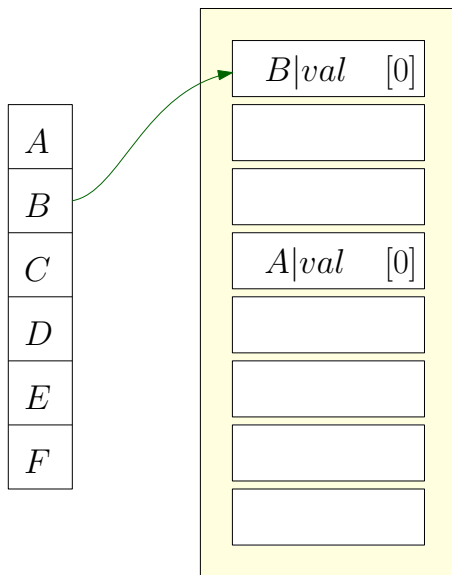
A
B
C
D
E
F



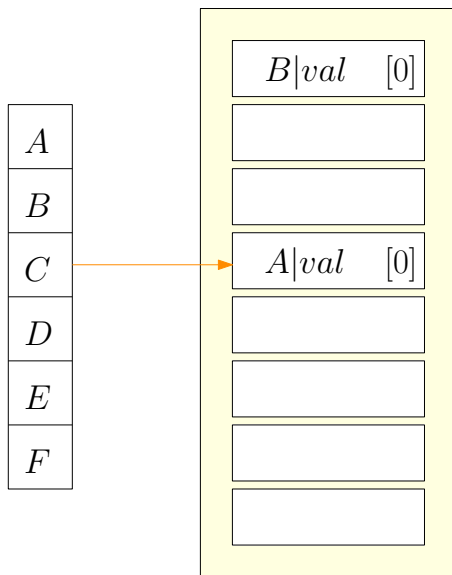
Próbkowanie liniowe Robin Hooda: wstawianie



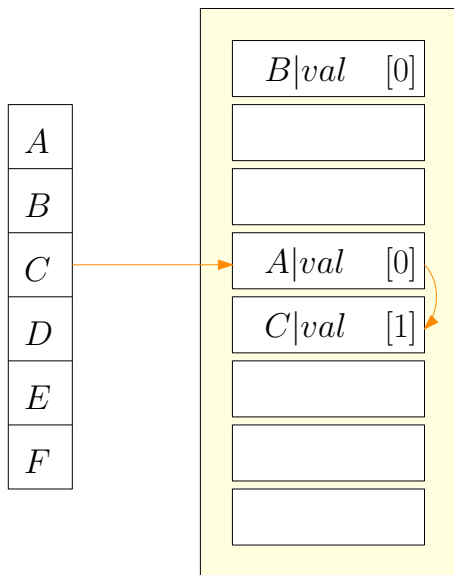
Próbkowanie liniowe Robin Hooda: wstawianie



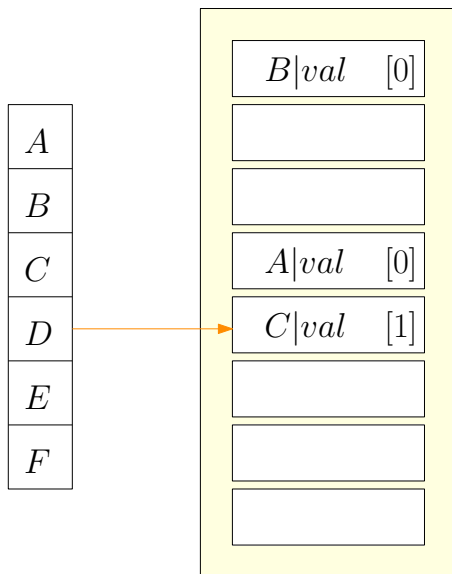
Próbkowanie liniowe Robin Hooda: wstawianie



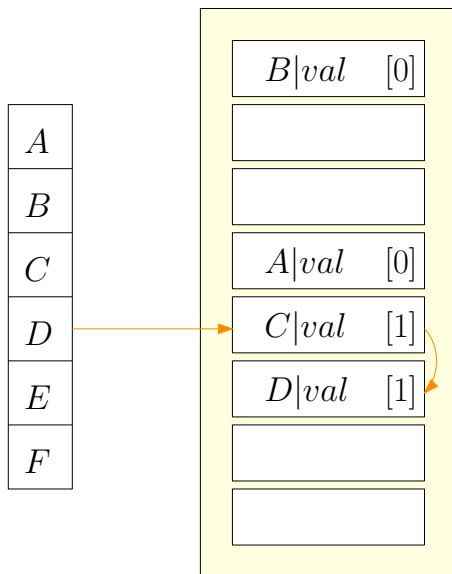
Próbkowanie liniowe Robin Hooda: wstawianie



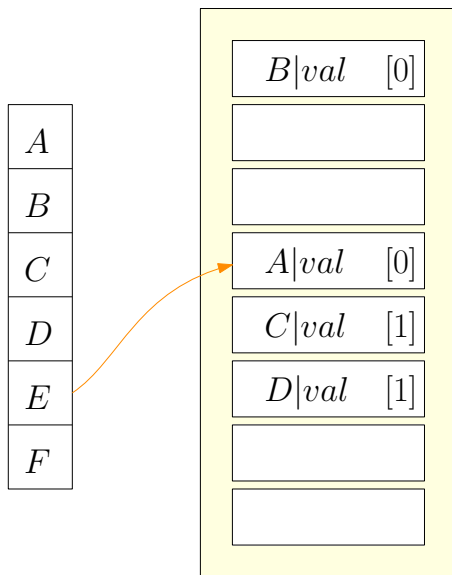
Próbkowanie liniowe Robin Hooda: wstawianie



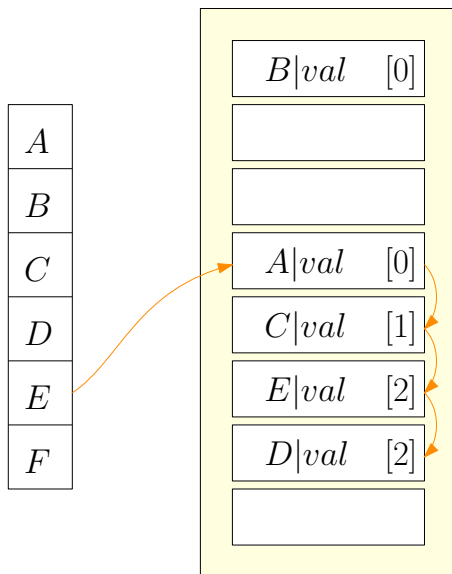
Próbkowanie liniowe Robin Hooda: wstawianie



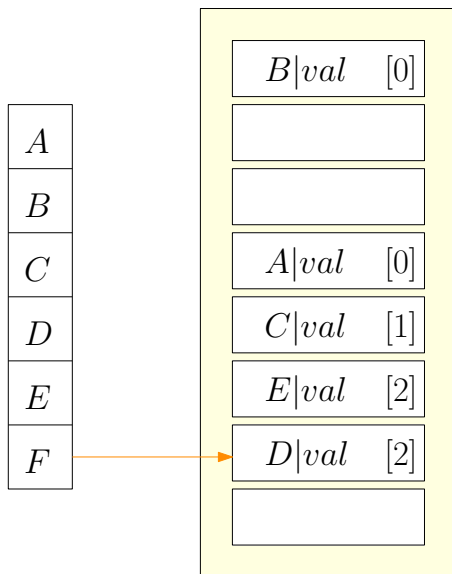
Próbkowanie liniowe Robin Hooda: wstawianie



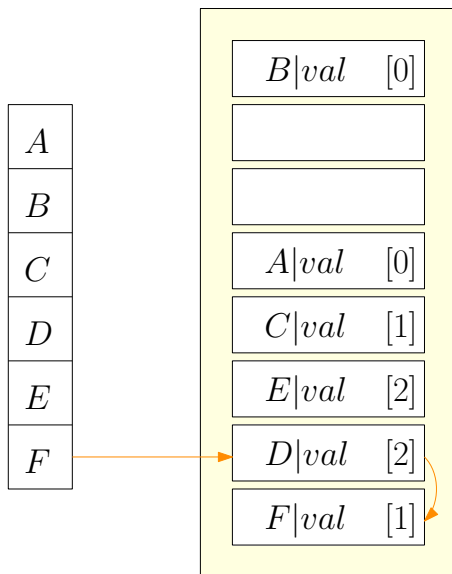
Próbkowanie liniowe Robin Hooda: wstawianie



Próbkowanie liniowe Robin Hooda: wstawianie



Próbkowanie liniowe Robin Hooda: wstawianie



Próbkowanie liniowe Robin Hooda: wstawianie

A
B
C
D
E
F

$B val$ [0]
$A val$ [0]
$C val$ [1]
$E val$ [2]
$D val$ [2]
$F val$ [1]

Haszowanie kukułcze

W przypadku **haszowania kukułczego** (*cuckoo hashing*) rozwiązujemy problem kolizji stosując dwie tablice i dwie odpowiadające im funkcje haszujące.

Haszowanie kukułcze

W przypadku **haszowania kukułczego** (*cuckoo hashing*) rozwiązujemy problem kolizji stosując dwie tablice i dwie odpowiadające im funkcje haszujące.

Dopóki nie występuje kolizja, dodajemy elementy do pierwszej tablicy zgodnie z wartościami pierwszej funkcji haszującej.

Jeśli dojdzie do kolizji, to wstawiamy dany element do drugiej tablicy na pozycji wyznaczonej przez drugą funkcję.

Jeśli i w tym przypadku natrafiamy na pewien element, to usuwamy go z drugiej tablicy i dla niego rekurencyjnie uruchamiamy procedurę wstawiania, przy czym tym razem wstawiamy go od razu do pierwszej tablicy.

Proces ten jest powtarzany do momentu, aż przestaną występować kolizje.

Haszowanie kukułcze

W przypadku **haszowania kukułczego** (*cuckoo hashing*) rozwiązujemy problem kolizji stosując dwie tablice i dwie odpowiadające im funkcje haszujące.

Dopóki nie występuje kolizja, dodajemy elementy do pierwszej tablicy zgodnie z wartościami pierwszej funkcji haszującej.

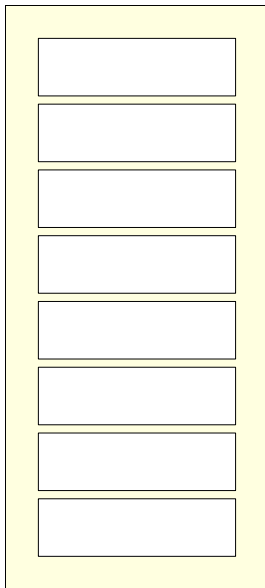
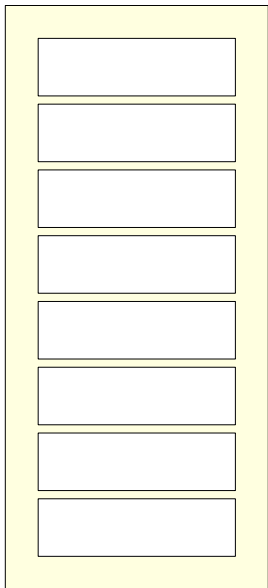
Jeśli dojdzie do kolizji, to wstawiamy dany element do drugiej tablicy na pozycji wyznaczonej przez drugą funkcję.

Jeśli i w tym przypadku natrafiamy na pewien element, to usuwamy go z drugiej tablicy i dla niego rekurencyjnie uruchamiamy procedurę wstawiania, przy czym tym razem wstawiamy go od razu do pierwszej tablicy.

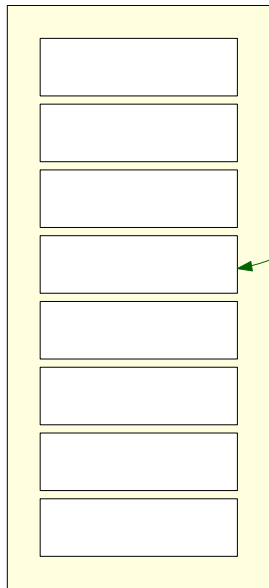
Proces ten jest powtarzany do momentu, aż przestaną występować kolizje.

Operacje odczytu i usuwania zawsze są wykonywane w czasie $\mathcal{O}(1)$.

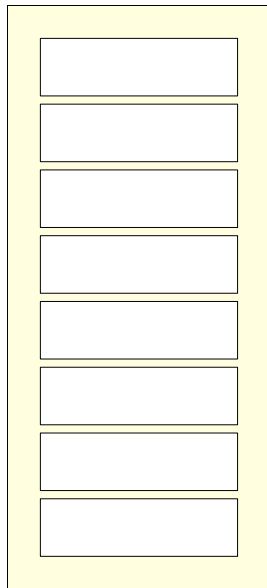
Haszowanie kukulcze: wstawianie



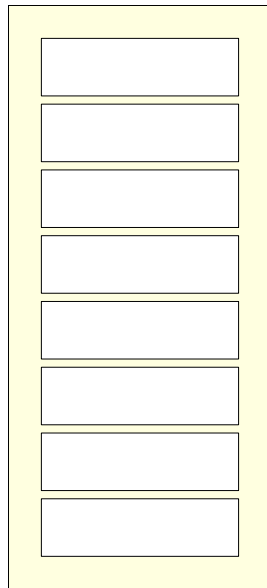
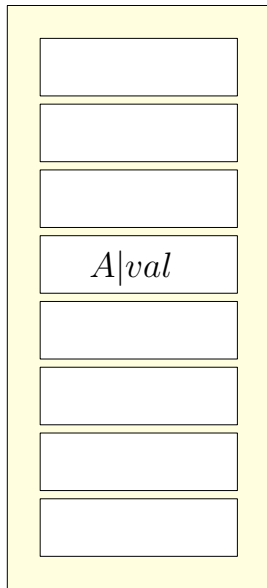
Haszowanie kukulcze: wstawianie



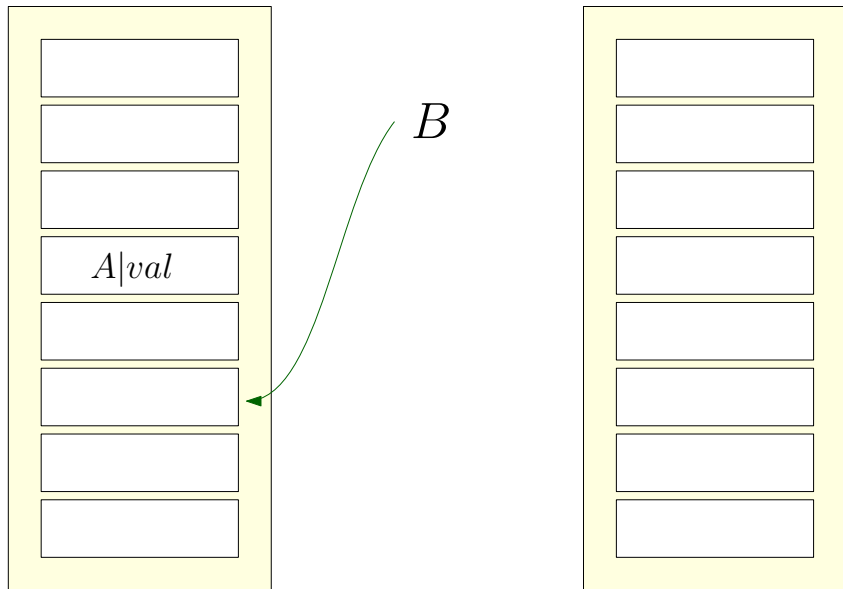
A



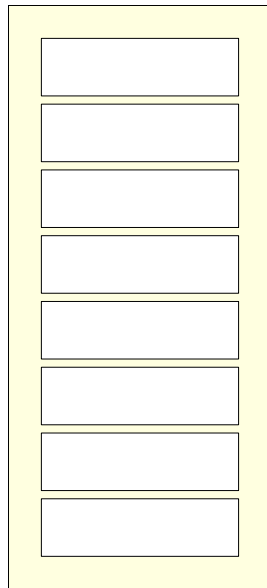
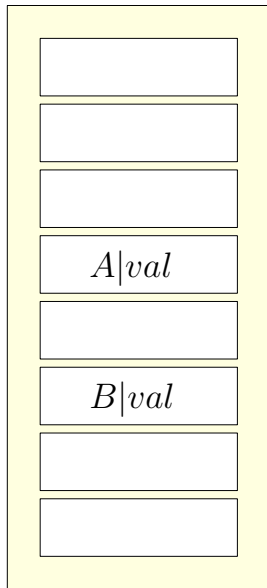
Haszowanie kukułcze: wstawianie



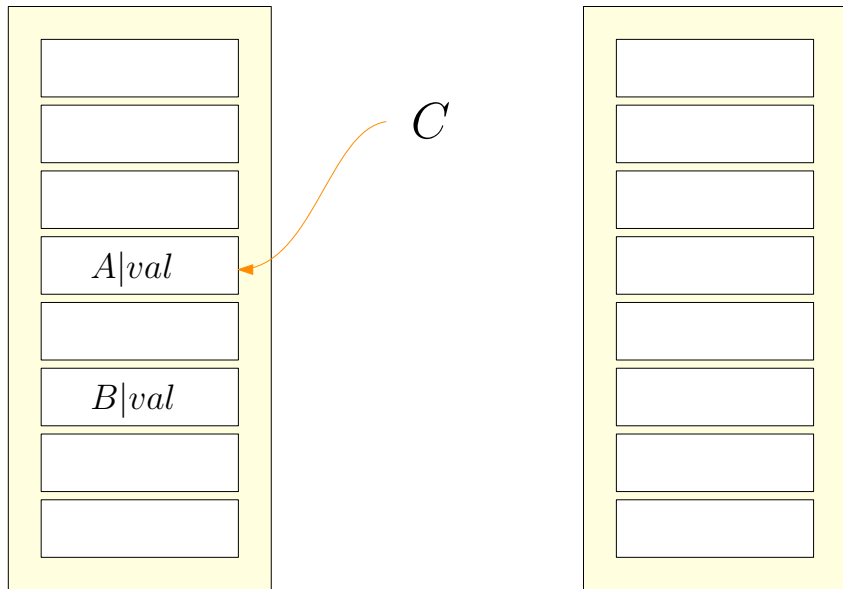
Haszowanie kukułcze: wstawianie



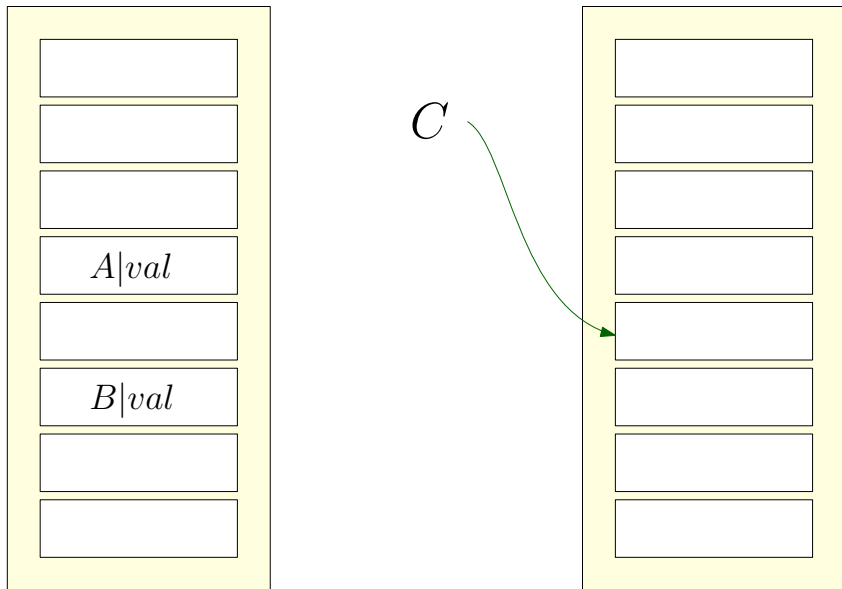
Haszowanie kukułcze: wstawianie



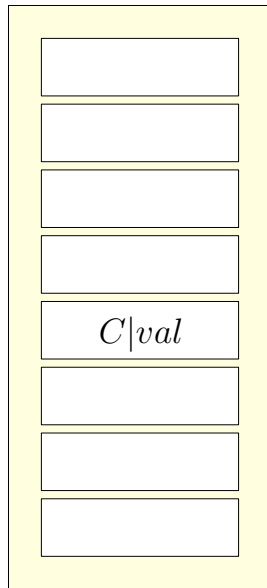
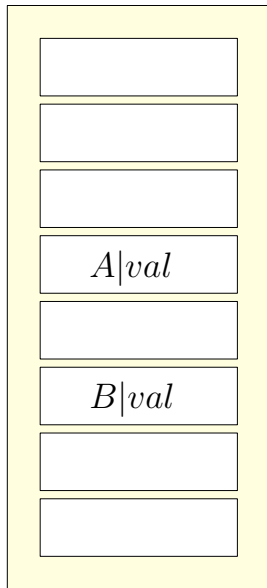
Haszowanie kukulcze: wstawianie



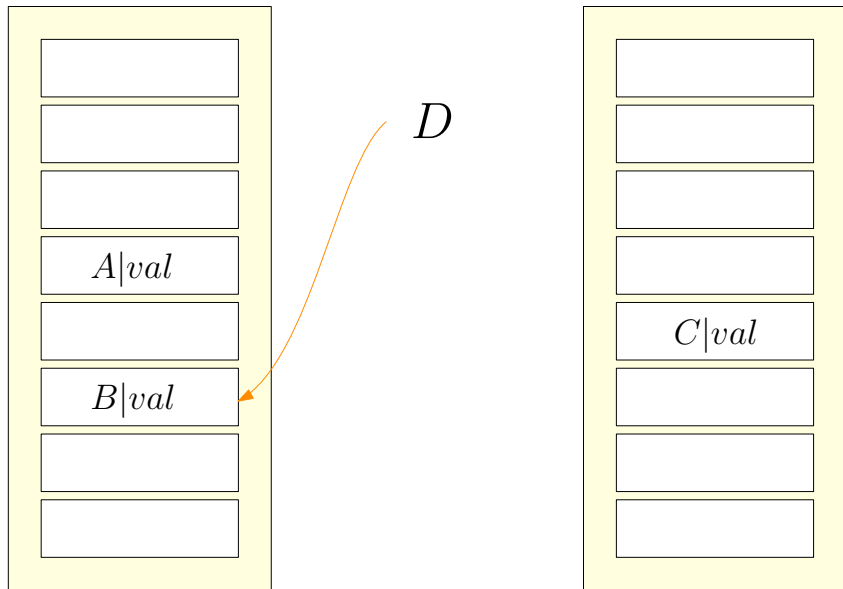
Haszowanie kukulcze: wstawianie



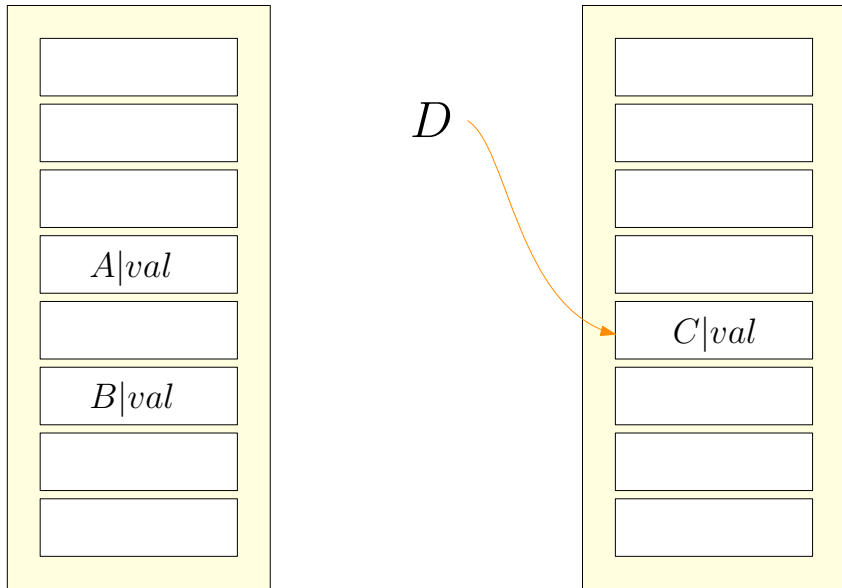
Haszowanie kukułcze: wstawianie



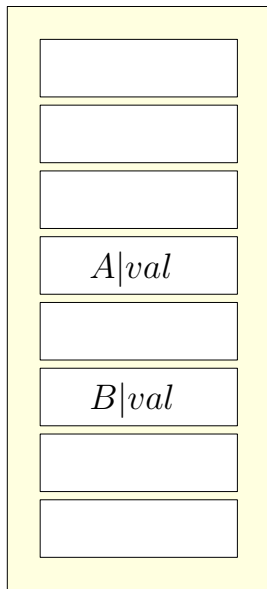
Haszowanie kukulcze: wstawianie



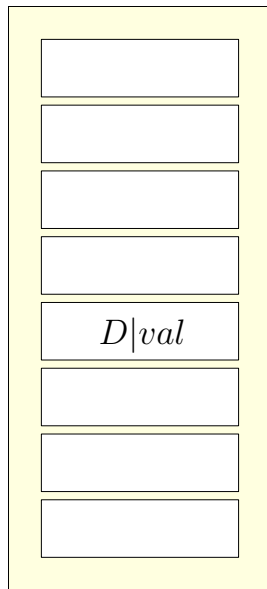
Haszowanie kukulcze: wstawianie



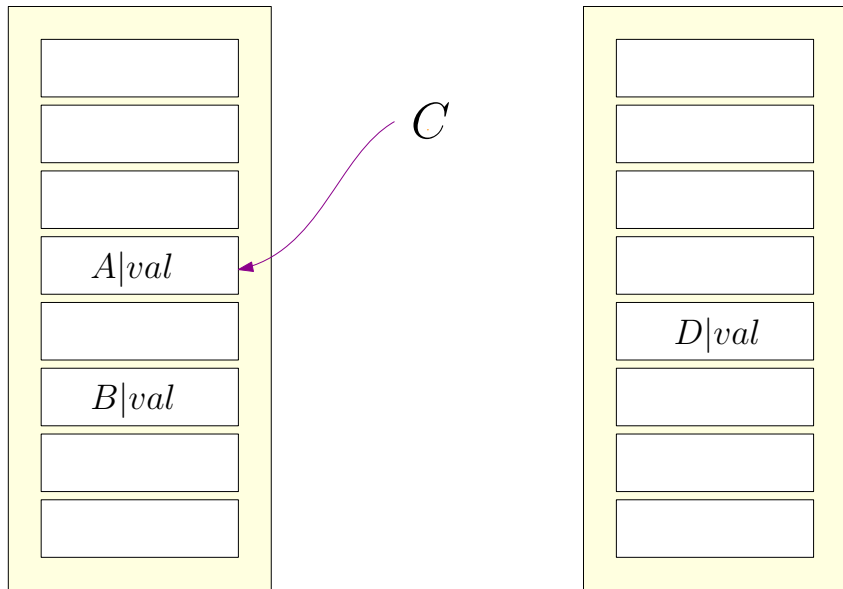
Haszowanie kukułcze: wstawianie



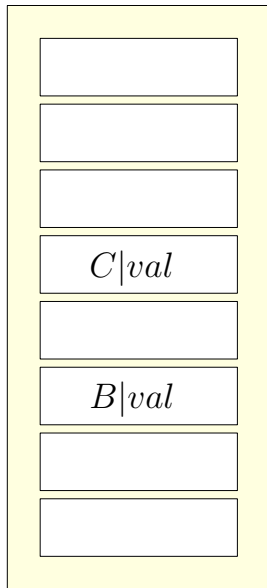
C



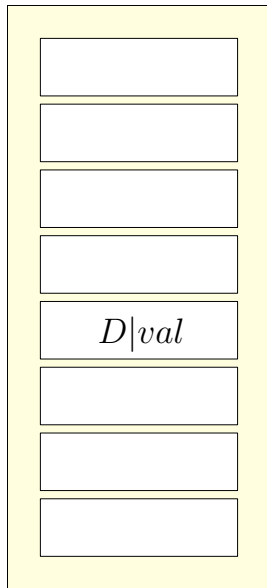
Haszowanie kukulcze: wstawianie



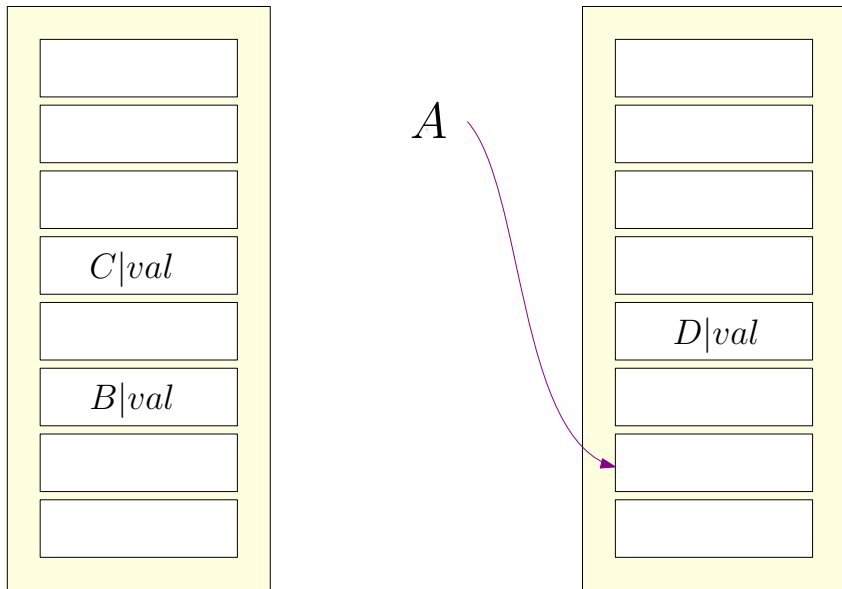
Haszowanie kukułcze: wstawianie



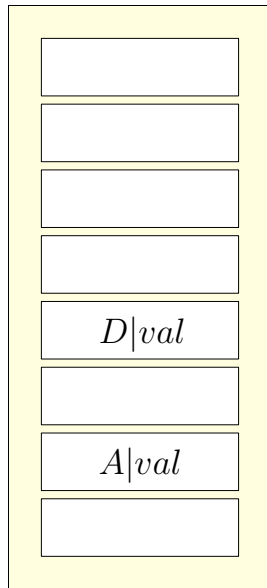
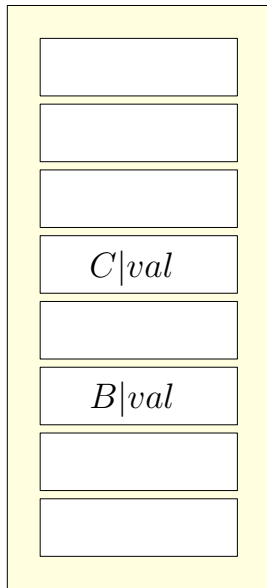
A



Haszowanie kukułcze: wstawianie



Haszowanie kukułcze: wstawianie



Wady haszowania statycznego

We wszystkich wcześniejszych przykładach SZBD musi wiedzieć, ile elementów ma przechowywać. Jednak większość baz danych rośnie z czasem. Jeśli już zdecydujemy się na haszowanie statyczne, to mamy trzy możliwe rozwiązania.

- Wybieramy funkcję haszującą na podstawie obecnego rozmiaru bazy. Wraz ze wzrostem bazy wydajność będzie spadać.
- Wybieramy funkcję haszującą dopasowaną do przewidywanego rozmiaru bazy. Skutkuje to początkowo niepotrzebnym zajęciem dostępnej pamięci.
- Co jakiś czas przeorganizujemy strukturę, dopasowując ją do rozmiaru bazy. Wymaga to wyboru nowej funkcji haszującej i obliczenia nowych wartości dla wszystkich rekordów. Taka operacja jest czasochłonna i wymaga zablokowania dostępu do bazy na czas jej wykonania.

W związku z tym warto rozważyć haszowanie dynamiczne.

Haszowanie rozszerzalne

Haszowaniu rozszerzalne (*extendible hashing*) radzi sobie ze zmianami rozmiaru bazy przez rozdzielanie (gdy baza rośnie) lub złączanie kubełków (gdy baza się zmniejsza).

Ponieważ taka operacja rozdzielania przeprowadzana jest tylko na jednym kubełku, jej koszt jest niewielki.

Haszowanie rozszerzalne

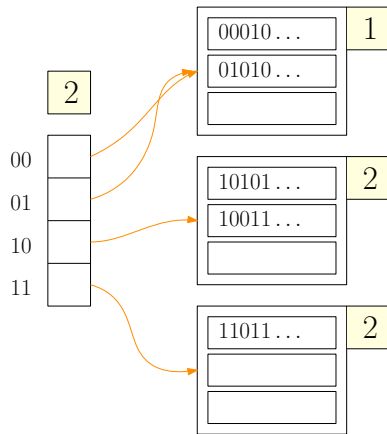
Haszowaniu rozszerzalne (*extendible hashing*) radzi sobie ze zmianami rozmiaru bazy przez rozdzielanie (gdy baza rośnie) lub złączanie kubełków (gdy baza się zmniejsza).

Ponieważ taka operacja rozdzielania przeprowadzana jest tylko na jednym kubełku, jej koszt jest niewielki.

Wybrana funkcja haszująca generuje liczby binarne o długości b (często $b = 32$), ma zatem bardzo dużo możliwych wartości.

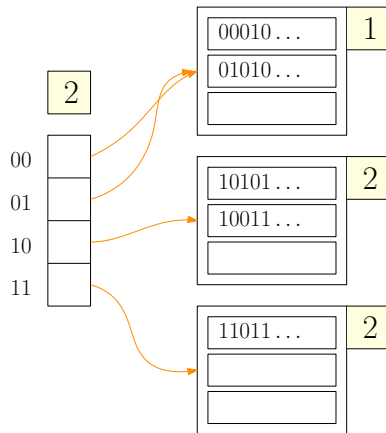
Nie tworzymy jednak kubełka dla każdej z nich, ani też (początkowo) nie wykorzystujemy wszystkich b bitów. Dla każdego kubełka j przechowujemy minimalną wartość $b_j \in \{0, \dots, b\}$ taką, że b_j pierwszych bitów jednoznacznie wyznacza ten kubełek. Zatem $\max_j b_j$ bitów wystarczy, aby jednoznacznie przypisać dany rekord do kubełka. Jeśli w kubełku j zabraknie miejsca, rozdzielamy go na dwa kubełki: j_1 i j_2 , gdzie $b_{j_1} = b_{j_2} = b_j + 1$.

Haszowanie rozszerzalne: wstawianie

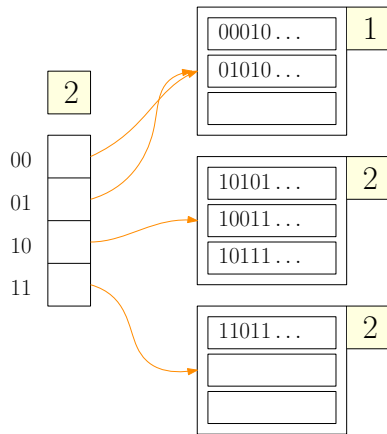


Haszowanie rozszerzalne: wstawianie

INSERT B : $\text{hash}(B) = 10111 \dots$

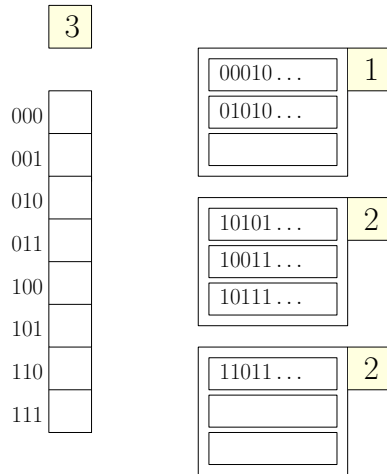


Haszowanie rozszerzalne: wstawianie

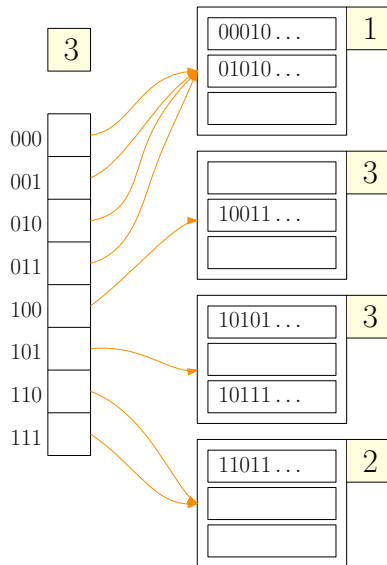


Haszowanie rozszerzalne: wstawianie

INSERT C : $\text{hash}(C) = 10100 \dots$



Haszowanie rozszerzalne: wstawianie



Haszowanie rozszerzalne: wstawianie

